

LABTRACK System Documentation

Hardware Asset Management System with App-Based Equipment Booking
and Defect Reporting

College of Computing Studies, Pampanga State University

Document type: Technical reference for thesis / capstone paper

Contents: Technology stack and database entity reference (for ERD)

Generated: August 24, 2026

Source: LABTRACK monorepo (supabase/migrations, application manifests)

1. System Overview

LABTRACK is a cloud-hosted asset management system composed of a **web administration dashboard**, an **Android mobile application**, and a **Supabase backend** (PostgreSQL database, authentication, file storage, row-level security, and RPC functions). Instructors scan QR codes on lab equipment to view details, request bookings, report defects with photos, and communicate with administrators through ticket-based messaging.

1.1 Application Components

Component	Path	Purpose
Web Admin Dashboard	apps/web	Super Admin, Admin/Custodian workflows: assets, QR codes, bookings, defects, tickets, access settings
Mobile App (Android)	apps/mobile	Faculty/Instructor/Borrower workflows: QR scan, booking, defect reporting, notifications, tickets
Shared Domain Package	packages/shared	Shared TypeScript types, Zod schemas, QR helpers, workflow constants
Database & Backend	supabase/migrations	PostgreSQL schema, RLS policies, storage buckets, RPC functions, cron jobs

1.2 User Roles

, custodian, faculty

Role	Enum Value	Primary Access
Super Admin	super_admin	User management, registration policy, university email domains, system settings
Admin / Custodian	admin	Asset catalog, QR generation, booking decisions, checkout/return, defect triage, tickets
Instructor / Faculty	instructor	Mobile: QR scan, booking requests, defect reports, ticket chat, notifications
Student	student	Extended role enum (reserved for future scope)

2. Technology Stack

2.1 General / Repository

Category	Technology	Version / Notes
Architecture	TypeScript monorepo (npm workspaces)	Root package name: labtrack
Runtime	Node.js	≥ 20.9.0
Package manager	npm	Root package-lock.json
Language	TypeScript	5.9.3 (web/shared), 6.0.3 (mobile)
Validation	Zod	4.1.13 (@labtrack/shared)
Version control	Git	—

2.2 Web Application (apps/web)

Category	Technology	Version
Framework	Next.js (App Router)	16.1.6
UI library	React	19.2.3
Backend client	@supabase/supabase-js	2.89.0
SSR auth integration	@supabase/ssr	0.7.0
Icons	lucide-react	0.468.0
QR code display	qrcode.react	4.2.0
Styling	Custom CSS (CSS variables, glass-style UI)	—
Server features	Next.js Server Actions	Body size limit: 6 MB
Deployment target	Vercel	Documented in architecture

2.3 Mobile Application (apps/mobile)

Category	Technology	Version
Framework	Expo SDK	56.0.5
Navigation	Expo Router	56.2.7
UI runtime	React Native	0.85.3
UI library	React	19.2.3
Backend client	@supabase/supabase-js	2.89.0
QR scanning	expo-camera	~56.0.7
Push notifications	expo-notifications	~56.0.14
Secure storage	expo-secure-store	~56.0.4
UI effects	expo-blur, expo-linear-gradient	SDK 56
Date/time picker	@react-native-community/datetimepicker	9.1.0
Safe area / screens	react-native-safe-area-context, react-native-screens	5.7.0 / 4.25.2
Dev client	expo-dev-client	~56.0.18
Platform target	Android (APK via EAS Build)	Android-first; iOS out of V1 scope
Build service	Expo Application Services (EAS)	CLI $\geq$ 16.0.0

2.4 Backend & Database (Supabase)

Category	Technology	Notes
Backend platform	Supabase (hosted)	Auth, Postgres, Storage, Realtime, PostgREST API
Database	PostgreSQL	Relational schema in public schema
Authentication	Supabase Auth	Email/password; signup domain restriction hook
Authorization	Row Level Security (RLS)	All public tables protected by RLS policies
File storage	Supabase Storage	Private buckets: defect-photos, asset-images
Business logic	PostgreSQL RPC functions	Booking, defect, QR, ticket, notification workflows
Extensions	pgcrypto, pg_cron	UUID generation; scheduled keepalive jobs
Private schema	app_private	Helper functions, keepalive audit table (not client-accessible)
Local development	Supabase CLI	supabase start, supabase db reset

2.5 Development & Build Tools

- **Testing:** Node.js built-in test runner (node --test) in packages/shared
- **Type checking:** tsc --noEmit across all workspaces

- **Android build scripts:** PowerShell scripts for emulator, dev-client install, APK build
(apps/mobile/scripts/)
- **Environment configuration:** .env.local (web), .env (mobile) with Supabase URL and publishable keys

3. Database Entities

The following tables are defined in Supabase PostgreSQL migrations. All tables in the `public` schema use Row Level Security unless noted. The `auth.users` table is managed by Supabase Auth and is linked to `public.profiles`.

3.1 Enumerated Types

Type Name	Values
<code>user_role</code>	<code>super_admin</code> , <code>admin</code> , <code>instructor</code> , <code>custodian</code> , <code>faculty</code> , <code>student</code>
<code>asset_status</code>	<code>available</code> , <code>reserved</code> , <code>checked_out</code> , <code>under_review</code> , <code>for_repair</code> , <code>retired</code>
<code>asset_condition</code>	<code>excellent</code> , <code>good</code> , <code>fair</code> , <code>defective</code> , <code>for_repair</code> , <code>retired</code>
<code>booking_status</code>	<code>pending</code> , <code>approved</code> , <code>rejected</code> , <code>cancelled</code> , <code>checked_out</code> , <code>returned</code>
<code>defect_status</code>	<code>pending</code> , <code>under_review</code> , <code>sent_for_repair</code> , <code>resolved</code> , <code>rejected</code>
<code>thread_subject_type</code>	<code>booking</code> , <code>defect_report</code>
<code>notification_type</code>	<code>booking_update</code> , <code>defect_update</code> , <code>ticket_message</code> , <code>system</code>
<code>borrowing_resource_type</code>	<code>asset</code> , <code>room</code>

3.2 Entity: profiles

Description: Application user profile linked 1:1 with Supabase Auth user.

Column	Type	Constraints / Notes
<code>id</code>	<code>uuid</code>	PK, FK → <code>auth.users(id)</code> ON DELETE CASCADE
<code>email</code>	<code>text</code>	NOT NULL, UNIQUE
<code>full_name</code>	<code>text</code>	NOT NULL
<code>role</code>	<code>user_role</code>	NOT NULL, default <code>instructor</code>
<code>department</code>	<code>text</code>	nullable
<code>is_active</code>	<code>boolean</code>	NOT NULL, default <code>true</code>
<code>created_at</code>	<code>timestamptz</code>	NOT NULL
<code>updated_at</code>	<code>timestamptz</code>	NOT NULL

3.3 Entity: asset_categories

Description: Classification of lab equipment (e.g., Laptop, Projector).

Column	Type	Constraints / Notes
id	uuid	PK
name	text	NOT NULL, UNIQUE
description	text	nullable
created_at	timestampz	NOT NULL

3.4 Entity: locations

Description: Physical rooms or lab locations; also used as reservable room resources.

Column	Type	Constraints / Notes
id	uuid	PK
name	text	NOT NULL, UNIQUE
description	text	nullable
location_type	text	NOT NULL, default room
capacity	integer	nullable
is_reservable	boolean	NOT NULL, default true
display_order	integer	NOT NULL, default 0
created_at	timestampz	NOT NULL

3.5 Entity: assets

Description: Individual trackable hardware items with property number and lifecycle state.

Column	Type	Constraints / Notes
id	uuid	PK
property_number	text	NOT NULL, UNIQUE
serial_number	text	nullable
name	text	NOT NULL
category_id	uuid	FK → asset_categories(id)
location_id	uuid	FK → locations(id)
condition	asset_condition	NOT NULL
status	asset_status	NOT NULL
notes	text	nullable
is_room_bound	boolean	NOT NULL, default false
is_active	boolean	NOT NULL, default true
lifecycle_state	text	NOT NULL, default active
archived_at	timestampz	nullable
archived_by	uuid	FK → profiles(id)
archive_reason	text	nullable
created_by	uuid	FK → profiles(id)
created_at, updated_at	timestampz	NOT NULL

3.6 Entity: asset_qr_codes

Description: QR code records tied to assets; only one active QR per asset.

Column	Type	Constraints / Notes
id	uuid	PK
asset_id	uuid	FK → assets(id) ON DELETE CASCADE
code	text	NOT NULL, UNIQUE
is_active	boolean	NOT NULL; unique partial index per asset
generated_by	uuid	FK → profiles(id)
generated_at	timestampz	NOT NULL
invalidated_at, invalidated_by	timestampz / uuid	nullable; FK invalidated_by → profiles

3.7 Entity: asset_images

Description: Metadata for asset photos stored in Supabase Storage (asset-images bucket).

Column	Type	Constraints / Notes
id	uuid	PK
asset_id	uuid	FK → assets(id) ON DELETE CASCADE
storage_path	text	NOT NULL, UNIQUE
alt_text	text	nullable
is_primary	boolean	NOT NULL; one primary per asset
uploaded_by	uuid	FK → profiles(id)
created_at	timestampz	NOT NULL

3.8 Entity: asset_lifecycle_events

Description: Audit trail of asset status/condition changes.

Column	Type	Constraints / Notes
id	uuid	PK
asset_id	uuid	FK → assets(id) ON DELETE CASCADE
actor_id	uuid	FK → profiles(id)
event_type	text	NOT NULL
from_status, to_status	asset_status	nullable
from_condition, to_condition	asset_condition	nullable
notes	text	nullable
metadata	jsonb	NOT NULL, default {}
created_at	timestampz	NOT NULL

3.9 Entity: bookings

Description: Borrowing/reservation requests for an asset or a room location.

Column	Type	Constraints / Notes
id	uuid	PK
resource_type	borrowing_resource_type	NOT NULL (asset or room)
asset_id	uuid	FK → assets(id); required when resource_type = asset
location_id	uuid	FK → locations(id); required when resource_type = room
instructor_id	uuid	FK → profiles(id)
requested_start_at	timestampz	NOT NULL
requested_end_at	timestampz	NOT NULL; must be after start
purpose	text	NOT NULL
status	booking_status	NOT NULL, default pending
decided_by	uuid	FK → profiles(id)
decided_at	timestampz	nullable
decision_notes	text	nullable
created_at, updated_at	timestampz	NOT NULL

3.10 Entity: booking_events

Description: Status transition history for bookings.

Column	Type	Constraints / Notes
id	uuid	PK
booking_id	uuid	FK → bookings(id) ON DELETE CASCADE
actor_id	uuid	FK → profiles(id)
from_status	booking_status	nullable
to_status	booking_status	NOT NULL
notes	text	nullable
created_at	timestampz	NOT NULL

3.11 Entity: defect_reports

Description: Equipment defect/issue reports submitted by instructors.

Column	Type	Constraints / Notes
id	uuid	PK
asset_id	uuid	FK → assets(id)
instructor_id	uuid	FK → profiles(id)
title	text	NOT NULL
description	text	NOT NULL
status	defect_status	NOT NULL, default pending
triaged_by	uuid	FK → profiles(id)
triaged_at	timestampz	nullable
resolution_notes	text	nullable
created_at, updated_at	timestampz	NOT NULL

3.12 Entity: defect_photos

Description: Photo evidence for defect reports; files stored in defect-photos bucket.

Column	Type	Constraints / Notes
id	uuid	PK
defect_report_id	uuid	FK → defect_reports(id) ON DELETE CASCADE
storage_path	text	NOT NULL
uploaded_by	uuid	FK → profiles(id)
created_at	timestampz	NOT NULL

3.13 Entity: ticket_threads

Description: Support conversation thread linked to exactly one booking or one defect report.

Column	Type	Constraints / Notes
id	uuid	PK
subject_type	thread_subject_type	NOT NULL
booking_id	uuid	FK → bookings(id) ON DELETE CASCADE; set when subject_type = booking
defect_report_id	uuid	FK → defect_reports(id) ON DELETE CASCADE; set when subject_type = defect_report
created_at	timestampz	NOT NULL

3.14 Entity: ticket_messages

Description: Individual messages within a ticket thread.

Column	Type	Constraints / Notes
id	uuid	PK
thread_id	uuid	FK → ticket_threads(id) ON DELETE CASCADE
sender_id	uuid	FK → profiles(id)
body	text	NOT NULL
created_at	timestampz	NOT NULL

3.15 Entity: notifications

Description: In-app notifications for booking, defect, ticket, and system events.

Column	Type	Constraints / Notes
id	uuid	PK
recipient_id	uuid	FK → profiles(id) ON DELETE CASCADE
type	notification_type	NOT NULL
title	text	NOT NULL
body	text	NOT NULL
read_at	timestampz	nullable
created_at	timestampz	NOT NULL

3.16 Entity: device_push_tokens

Description: Expo push notification tokens registered by mobile users.

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK → profiles(id) ON DELETE CASCADE
expo_push_token	text	NOT NULL, UNIQUE
platform	text	NOT NULL, default android
created_at	timestampz	NOT NULL
last_seen_at	timestampz	NOT NULL

3.17 Entity: audit_logs

Description: Administrative action audit trail.

Column	Type	Constraints / Notes
id	uuid	PK
actor_id	uuid	FK → profiles(id)
action	text	NOT NULL
entity_table	text	NOT NULL
entity_id	uuid	nullable
metadata	jsonb	NOT NULL, default {}
created_at	timestampz	NOT NULL

3.18 Entity: university_email_domains

Description: Allowed email domains for self-registration.

Column	Type	Constraints / Notes
id	uuid	PK
domain	text	NOT NULL, UNIQUE (e.g., pampangastateu.edu.ph)
is_allowed	boolean	NOT NULL, default true
notes	text	nullable
created_at	timestampz	NOT NULL

3.19 Entity: registration_settings

Description: Singleton configuration row for signup domain restriction policy.

Column	Type	Constraints / Notes
id	boolean	PK, always true (singleton)
restrict_signup_to_allowed_domains	boolean	NOT NULL, default true
created_at, updated_at	timestampz	NOT NULL

3.20 Internal Entity: app_private.supabase_keepalive_events

Description: Internal cron audit table (not exposed to client apps). Used by pg_cron scheduled jobs.

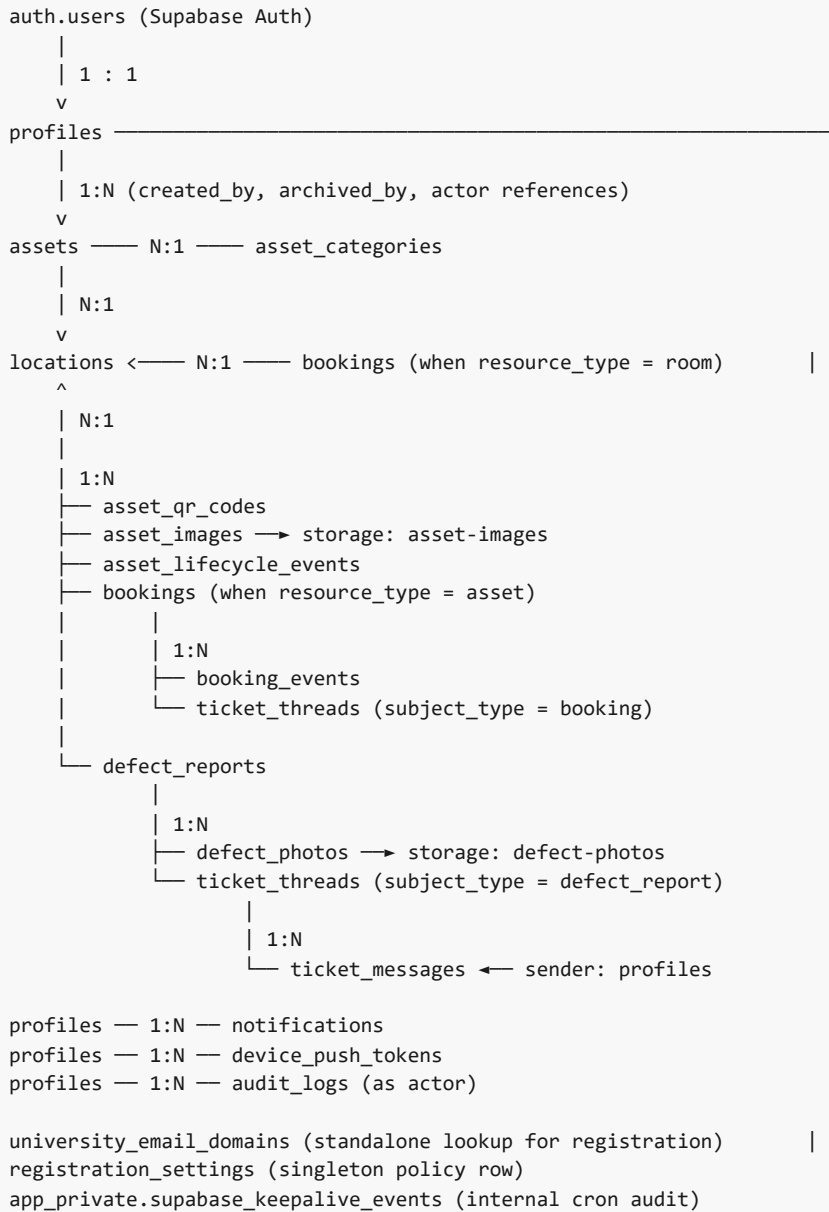
Column	Type	Constraints / Notes
id	bigint	PK, identity
ran_at	timestampz	NOT NULL
note	text	NOT NULL

3.21 Storage Buckets (Supabase Storage)

Bucket ID	Access	Linked Entity
defect-photos	Private	defect_photos.storage_path
asset-images	Private	asset_images.storage_path

4. Entity Relationship Diagram (Reference)

Use this diagram as a reference when drawing the formal ERD for your paper. Cardinality notation: **1** = one, **N** = many, **0..1** = optional one.



4.1 Relationship Summary Table

Parent Entity	Child Entity	Relationship	FK Column
auth.users	profiles	1 : 1	profiles.id
asset_categories	assets	1 : N	assets.category_id
locations	assets	1 : N	assets.location_id
assets	asset_qr_codes	1 : N (1 active)	asset_qr_codes.asset_id
assets	asset_images	1 : N	asset_images.asset_id
assets	asset_lifecycle_events	1 : N	asset_lifecycle_events.asset_id
assets	bookings	1 : N	bookings.asset_id
locations	bookings	1 : N	bookings.location_id
profiles	bookings	1 : N	bookings.instructor_id
bookings	booking_events	1 : N	booking_events.booking_id
assets	defect_reports	1 : N	defect_reports.asset_id
profiles	defect_reports	1 : N	defect_reports.instructor_id
defect_reports	defect_photos	1 : N	defect_photos.defect_report_id
bookings	ticket_threads	1 : 0..1	ticket_threads.booking_id
defect_reports	ticket_threads	1 : 0..1	ticket_threads.defect_report_id
ticket_threads	ticket_messages	1 : N	ticket_messages.thread_id
profiles	ticket_messages	1 : N	ticket_messages.sender_id
profiles	notifications	1 : N	notifications.recipient_id
profiles	device_push_tokens	1 : N	device_push_tokens.user_id
profiles	audit_logs	1 : N	audit_logs.actor_id

5. Key Backend RPC Functions (Business Logic)

Critical workflows are enforced through PostgreSQL RPC functions rather than direct table writes:

- `resolve_asset_by_qr_code` — QR-scoped asset lookup for mobile users
- `regenerate_asset_qr` — Atomic QR replacement with audit logging
- `create_booking`, `cancel_booking`, `decide_booking`, `checkout_booking`, `return_booking` — Booking lifecycle
- `create_defect_report`, `triage_defect_report` — Defect workflow
- `ensure_ticket_thread`, `send_ticket_message` — Ticket messaging
- `mark_notification_read` — Notification read state
- `hook_restrict_signup_by_email_domain` — Auth hook for domain-restricted registration

This document was generated from the LABTRACK source repository. For the latest schema, refer to [supabase/migrations/](#).
Total public entities: 19 tables + 2 storage buckets + Supabase Auth users.